

Proyecto final: Laberinto inteligente

Grupo:

Juan José Agudelo Muñoz
Juan Pablo Montoya Rivera
Emanuel Henao Echeverry

Junio 19 de 2026

Profesora: Carlos Manuel Orrego Franco

Asignatura: Introducción a ciencias de la computación

Institución: Universidad Nacional de Colombia Sede Manizales

Entendiendo el algoritmo

Nuestro equipo se dedicó a estudiar la estructura del código base entregado para la asignatura de introducción a ciencias de la computación. Se analizó la representación del laberinto en las listas y se agregaron comentarios explicando los símbolos que conforman el laberinto: "S.es el inicio, "G.es la meta, "."significa un camino donde se puede pasar y "#"son los muros. Se revisaron las tres posibilidades de prueba que serían: (laberinto_con_solucion, laberinto_sin_solucion y laberinto_varios_caminos). Estas 3 posibilidades fueron fundamentales para el grupo para analizar cómo funciona el código ante diferentes niveles.

Entender las Restricciones del código

Nosotros nos enfocamos en documentar cómo el código valida los movimientos permitidos dentro de la matriz. Se comentó la función obtener_vecinos, que determina las cuatro coordenadas adyacentes posibles (arriba, abajo, izquierda, derecha). Se documentó la función es_valido. El grupo decidió que es importante entenderla, ya que evita tres errores: salirse de la matriz (con la función esta_dentro), intentar atravesar un muro ("#"), o entrar en un bucle infinito al visitar una coordenada que ya se encuentra en "visitados".

Adiciones del informe: El grupo definió el conjunto de celdas visitados como la estrategia de "poda" del algoritmo. Esta restricción es fundamental ya que evita que el algoritmo entre en ciclos recorriendo una y otra vez los mismos lugares, lo que reduce el número de caminos que se necesitan revisar.

Análisis del Algoritmo de Búsqueda y Backtracking

Nosotros analizamos el núcleo del código dentro de la función `resolver_laberinto`. Se añadieron comentarios sobre cómo la variable `camino` funciona para almacenar la ruta. Se estudió la función recursiva `backtracking`. Se identificó el comportamiento cuando el algoritmo llega a un callejón sin salida: ejecuta `camino.pop()` y remueve la celda de los visitados para eliminar el paso. El grupo concluyó que esta capacidad de no olvidar los errores y retroceder es lo que le da inteligencia a la búsqueda.

Adiciones del informe: El equipo logró identificar cada componente del `backtracking` dentro del código: la "solución parcial" es la lista `camino`, los "candidatos" son los vecinos generados, la "solución completa" se da cuando la posición actual es igual con la celda `G`, y el retroceso ocurre cuando se quita la última decisión eliminando posiciones de la lista y del conjunto de visitados.

Evaluación y Laberintos sin Solución

Al final el grupo analizó cómo el código evalúa su propio rendimiento y cómo maneja los casos imposibles. Se revisó el diccionario de estadísticas dentro del código base, el cual ve las llamadas recursivas, ramas descartadas y retrocesos. [cite: 34] Al correr la opción del `laberinto_sin_solucion`, y el sistema lanzó `False` tras almacenar 24 retrocesos.

Adiciones del informe: Durante las pruebas documentadas en el informe, el equipo observó que, en el laberinto con múltiples caminos válidos, se generó un número más grande de llamadas recursivas a comparación con el primer caso, lo que demostró que el código tuvo que ver muchas más rutas antes de terminar.

Conclusión del análisis: Ahora que nosotros entendemos al 100% el código del profesor, se puede explicar cómo el programa decide que un laberinto no tiene solución. Al evaluar el `laberinto_sin_solucion`, el código agota exhaustivamente todas las rutas posibles. Al visitar repetidamente con muros o celdas ya visitadas, el sistema hace `backtracking` constantemente hasta que todas las opciones se descartan. Al no lograr alcanzar la meta ("G"), la función finaliza y retorna `False`, demostrando que no hay ninguna ruta que nos haga llegar a la meta.

Reflexión Final

Para finalizar, nosotros como grupo reflexionamos que este proyecto fue una forma muy buena para aprender el concepto de `backtracking` de una manera más práctica. En general, se demostró que el algoritmo cumple todos los objetivos: encuentra un camino válido, identifica las matrices sin solución, marca los recorridos y registra las estadísticas de ejecución.